MediCheck

AI-Powered Healthcare Bill Accuracy & Dispute Assistant

Quantic MSSE Capstone | RAG + OCR + NLP | Microservices on Render | 12 Weeks | 2-Person Team

V2. Add microservices architecture
V3. Add geographic scope and targe insurers and service providers
V4. Adjust Sprint split. Sprint 1-5 2wks each. Sprint6 Final prep

1. PROBLEM STATEMENT

Approximately 80% of US medical bills contain errors, costing patients an estimated $68 billion annually in overcharges. The US healthcare billing system is structurally complex: a single outpatient procedure can generate multiple bills from separate providers — hospital, surgeon, anaesthesiologist, radiologist — each using different formats, alongside a separate Explanation of Benefits (EOB) from the patient's insurer.

Patients lack the domain knowledge to verify billing accuracy, detect duplicate charges, identify No Surprises Act violations, or reconcile provider bills against their EOB. Professional medical bill auditors charge $500–$2,000 upfront. No accessible, affordable consumer tool exists to fill this gap.

Concrete example: A patient in Texas receives an $18,400 hospital bill and a $3,200 anaesthesiologist bill after knee surgery, plus an EOB from her insurer. She cannot know the anaesthesiologist was out-of-network (a potential No Surprises Act violation), that a surgical tray was billed twice ($480 duplicate), or that anaesthesia time was upcoded by 90 minutes ($1,100 overcharge). She pays in full. MediCheck detects all three errors and generates a ready-to-send dispute letter.

2. PROPOSED SOLUTION

MediCheck is a web-based document analysis platform that combines OCR, natural language processing, and retrieval-augmented generation (RAG) to analyse medical bills and EOBs. Users upload documents, confirm extracted fields, receive a structured error report within 30 seconds, and download a ready-to-send dispute letter — all without medical billing expertise.

The system is built as a three-service microservices architecture deployed on Render's free tier, with bounded contexts drawn from domain-driven design principles. Each service is independently deployable, has its own responsibility, and communicates via defined REST APIs.

Architecture decision: Microservices are deployed on Render rather than AWS to keep operational complexity manageable within a 12-week part-time schedule. Render natively supports multiple independent services from a single GitHub repository, each with its own URL and environment. This delivers genuine microservices architecture — independent deployability, service isolation, defined inter-service APIs — without IAM, VPC, or container registry configuration overhead. The design document includes an AWS equivalent architecture diagram for comparison.

3. THREE-SERVICE MICROSERVICES ARCHITECTURE

MediCheck is split into three independently deployed services, each aligned to a distinct bounded context identified through domain-driven design:

Service	Name	Bounded Context	Render Type	URL
Service 1	Frontend	User Interface	Static Site (free)	medicheck.onrender.com
Service 2	Bill Analysis API	Bill Processing Domain	Web Service (free)	medicheck-api.onrender.com
Service 3	RAG & Letter Service	Knowledge & Explanation Domain	Web Service (free)	medicheck-rag.onrender.com

Service 1 — React Frontend (User Interface Context)

A React + Vite single-page application deployed as a Render static site. Handles all user interaction: document upload, field confirmation UI, error report display, and dispute letter download. Communicates exclusively with Service 2 via REST API calls. Has no direct knowledge of the RAG pipeline or database. Independently deployable — UI changes never require restarting backend services.

Service 2 — Bill Analysis API (Bill Processing Domain)

A Flask REST API handling document ingestion, OCR coordination, all four error detection modules, session management, and result storage. Calls AWS Textract for OCR. Persists session and results data to a Render-managed PostgreSQL database. When error detection is complete, calls Service 3 via internal HTTP to retrieve RAG-generated explanations. Returns structured JSON to Service 1. This service is the system's stateful core — it owns all user session data.

Service 3 — RAG & Letter Generation Service (Knowledge & Explanation Domain)

A Flask microservice responsible exclusively for knowledge retrieval and natural language generation. Hosts the ChromaDB vector store populated from the four free knowledge base sources. Receives structured error payloads from Service 2, retrieves relevant passages, and returns plain-English explanations with regulatory citations. Also generates dispute letter content on request. Stateless — no user session data. Independently scalable and replaceable without affecting error detection logic.

> **Why this split is architecturally correct:** Service 2 is deterministic and stateful. Service 3 is probabilistic and stateless. These are genuinely different scaling profiles, failure modes, and domain responsibilities — not an arbitrary split. This boundary maps directly to the DDD bounded contexts and satisfies the Project 3 microservices architecture requirement.

4. SERVICE COMMUNICATION & API DESIGN

Caller	Target	Endpoint	Method	Purpose
Service 1 (Frontend)	Service 2 (Analysis API)	POST /upload	REST/HTTP	Upload PDF documents
Service 1 (Frontend)	Service 2 (Analysis API)	POST /confirm	REST/HTTP	Submit confirmed extracted fields
Service 1 (Frontend)	Service 2 (Analysis API)	POST /analyse	REST/HTTP	Trigger error detection pipeline
Service 1 (Frontend)	Service 2 (Analysis API)	GET /report/{id}	REST/HTTP	Retrieve analysis results

Service 1 (Frontend)	Service 2 (Analysis API)	POST /letter	REST/HTTP	Request dispute letter generation
Service 2 (Analysis API)	Service 3 (RAG Service)	POST /explain	Internal REST	Get RAG explanations for flagged errors
Service 2 (Analysis API)	Service 3 (RAG Service)	POST /draft-letter	Internal REST	Generate dispute letter content
CI/CD Pipeline	All services	GET /health	HTTP smoke test	Post-deploy health verification

5. DOMAIN-DRIVEN DESIGN

MediCheck applies domain-driven design principles to identify service boundaries and shared language. Two core bounded contexts drive the architecture:

Bill Processing Domain (Service 2)

- Core entities: Bill, EOBDocument, LineItem, DetectionResult, Session, DisputeLetter
- Value objects: CPTCode, ServiceDate, BilledAmount, MedicareRate, ProviderType
- Aggregates: BillAnalysis (root) — owns LineItems and DetectionResults
- Ubiquitous language: 'duplicate charge', 'rate outlier', 'EOB mismatch', 'NSA violation', 'confirmed fields', 'analysis session'

Knowledge & Explanation Domain (Service 3)

- Core entities: KnowledgeChunk, Citation, ExplanationResult, LetterDraft
- Value objects: EmbeddingVector, RetrievalScore, GroundednessScore
- Aggregates: RAGResponse (root) — owns Citations and ExplanationResult
- Ubiquitous language: 'grounded explanation', 'citation', 'knowledge base', 'retrieval chunk', 'regulatory reference', 'letter draft'

6. FOUR MVP ERROR DETECTION MODULES

Design pattern: All four modules implement the Strategy pattern via a BaseDetector abstract class. The ErrorDetectionEngine in Service 2 accepts any list of BaseDetector implementations and aggregates results. Adding a fifth detector requires zero changes to existing code.

Module	Error Type	Detection Logic	Data Required
1	Duplicate charge	Same CPT code on same date of service — structural comparison, no CPT descriptions needed	Line items: CPT code, service date
2	Medicare rate outlier	Billed amount exceeds 300% of CMS Physician Fee Schedule locality-adjusted rate	CMS fee schedule CSV, CPT code, locality, billed amount
3	EOB reconciliation	Cross-reference line items between provider bill and insurer EOB; flag mismatches in codes, dates, quantities, amounts	Both documents' confirmed line items

4	No Surprises Act violation	Out-of-network provider flag in EOB context where NSA (2022) prohibits balance billing	EOB provider network status, service context

7. RAG KNOWLEDGE BASE

Source	Content	Licensing	Format
CMS Physician Fee Schedule	Medicare reimbursement rates by CPT code and locality. Updated annually.	Free — public domain	CSV from cms.gov
ICD-10-CM Guidelines	Diagnosis coding conventions for upcoding detection.	Free — CMS/CDC	PDF from cms.gov
No Surprises Act & CMS Guidance	Federal law text and out-of-network billing protections.	Free — public domain	Text from congress.gov
CMS Procedure-to-RVU Crosswalk	Procedure groupings used without AMA-copyrighted CPT descriptions.	Free — CMS	Excel from cms.gov

CPT licensing: Full CPT code descriptions are copyrighted by the AMA (~$1,000-2,500/yr commercial license). All four detection modules operate on CPT code numbers and CMS rate data only — no copyrighted text descriptions are used or stored. Documented as an explicit architectural decision.

8. TECHNOLOGY STACK

Layer	Technology	Service	Rationale
Frontend framework	React + Vite	Service 1	Team's existing skill; clean component architecture for multi-step flow
Frontend styling	Tailwind CSS + Flexbox/Grid	Service 1	Responsive, consistent UI; satisfies Project 2 CSS layout requirement
Bill Analysis API	Flask (Python)	Service 2	Team's existing skill; lightweight REST API; Python ML/NLP ecosystem
RAG & Letter API	Flask (Python)	Service 3	Consistent with Service 2; LangChain runs natively in Python
OCR	AWS Textract (free tier)	Service 2	Handles digital and scanned PDFs; structured layout extraction
Database	PostgreSQL (Render managed)	Service 2	Session and results persistence; team's existing skill
Vector store	ChromaDB (embedded)	Service 3	Lightweight, open-source, no managed service cost; prior experience
Embeddings	OpenAI text-embedding-3-small	Service 3	Low cost (~$0.02/1M tokens); high quality for medical text

LLM	GPT-4o-mini via OpenAI API	Service 3	Cost-effective; strong instruction-following; temperature=0 for determinism
RAG framework	LangChain (LCEL)	Service 3	Team has prior experience; mature document Q&A tooling
Letter export	python-docx + ReportLab	Service 2	Word and PDF dispute letter generation and download
Deployment	Render (free tier)	All services	Three independent services from one GitHub repo; HTTPS; no config overhead
Version control	GitHub	All	Shared repo; shared with quantic-grader; PR-based workflow
CI/CD	GitHub Actions	All	Pytest + lint on PR; auto-deploy all three Render services on merge to main
Task board	Trello	All	Product backlog, sprint tracking, completion status per Capstone rubric
Wireframes	Figma (free tier)	Design	Industry-standard; satisfies Project 1 wireframe requirement

9. 11-WEEK SPRINT PLAN (PART-TIME, 2-PERSON TEAM)

Five sprints of 2 weeks each, plus one final sprint of 1 week — 11 weeks total. Both members hold full-time employment; AI coding assistants accelerate throughput. A sprint demo recording is submitted to the Product Owner at the end of each sprint.

Sprint 1 is process and design only — no code. This ensures requirements, user stories, and wireframes are complete before any infrastructure is built, so Sprint 2 code is purposeful from the first line. Sprint 6 is a dedicated 1-week presentation sprint — all code is frozen at the end of Sprint 5.

Sprint	Weeks	Goals	Rubric Deliverable
Sprint 1	1–2	PROCESS & DESIGN ONLY — no code written. GitHub repo created and shared with quantic-grader. Trello board fully populated with all user stories before any coding begins. 15+ user stories from three personas. Complete user journey map. 20+ system requirements with traceability matrix. Figma wireframes for three core screens with requirement ID annotations. Team roles documented. README.md skeleton and .env.example committed.	Repo live (empty), shared with quantic-grader. Trello board fully populated. Sprint 1 demo: walkthrough of Trello board, user stories, and Figma wireframes.
Sprint 2	3–4	INFRASTRUCTURE & FOUNDATION. Three Render services scaffolded (minimal Flask + React shells). PostgreSQL schema designed and provisioned on Render. /health endpoint on both Flask services returning 200. GitHub Actions CI/CD pipeline: pytest + lint on PR; auto-deploy all three services on merge to main; post-deploy /health smoke test. Service 2: AWS Textract OCR integration. Field confirmation React UI. Service-to-service URL wired via environment variable.	Repo with working code. Three Render URLs live, /health responding. CI/CD pipeline shown running in GitHub Actions. Sprint 2 demo: upload PDF → Textract extracts fields → field confirmation UI shown.
Sprint 3	5–6	KNOWLEDGE BASE & ERROR DETECTION. Service 3: ChromaDB knowledge base ingested and queryable. LangChain RAG chain returning cited responses. All four error detection modules implemented with Strategy pattern (BaseDetector, ErrorDetectionEngine). Structured JSON results from POST /analyse. Error report results UI in Service 1.	Sprint 3 demo: upload bill with known errors → all four modules flag correctly → error report with colour-coded results shown in browser. RAG query returns cited answer from knowledge base.
Sprint 4	7–8	END-TO-END FLOW & DISPUTE LETTERS. Service 3: RAG explanation chain producing grounded explanations with citations wired into results payload. Service 3: dispute letter generation (POST /draft-letter). Service 2: calls Service 3, formats via python-docx and ReportLab. Service 1: Word and PDF download buttons. Full end-to-end flow working: upload → confirm → analyse → explained report → dispute letter download.	Sprint 4 demo: complete end-to-end flow with real sample document. RAG-grounded explanations shown with citations. Word and PDF dispute letter downloaded on camera.
Sprint 5	9–10	TESTING, EVALUATION & DOCUMENTATION. Comprehensive test suite: 15+ unit tests across all modules,	Sprint 5 demo: test suite run shown passing in CI. RAG evaluation results documented. All Trello stories

		integration tests, post-deploy smoke tests. RAG evaluation: 15-20 question set, groundedness and citation accuracy scored, latency measured. Design and testing document: UML class and sequence diagrams, Strategy pattern documented, anti-patterns resolved, bounded context diagram, microservices architecture diagram, deployment cost analysis. All Trello stories marked done. Code documentation pass. CODE FREEZE at end of Sprint 5.	complete. Design and testing doc committed to repo.
Sprint 6	11	PRESENTATION WEEK — no new code. Final presentation rehearsal using pre-prepared sample documents. Verify all three Render services warm and responding. Record 15-20 minute Zoom presentation: both members on camera, government IDs shown, full end-to-end demo, CI/CD pipeline shown running. Upload to YouTube unlisted. Submit to Quantic dashboard.	Final recorded presentation (15-20 min) submitted. ai-tooling.md completed. All rubric deliverables verified and submitted.

10. TEAM ROLES (2-PERSON TEAM)

Role	Member	Primary Responsibilities
Product Owner + Developer	Member 1	Owns user stories, Trello backlog, acceptance criteria, and Figma wireframes. Leads RAG chain design (Service 3), LangChain integration, LLM prompt engineering, knowledge base ingestion, ChromaDB setup, RAG evaluation, and dispute letter generation. Drives final presentation narrative. Contributes equally to all development.
Scrum Master + Code Owner + Developer	Member 2	Facilitates standups and sprint planning. Owns GitHub repo and approves all PRs to main. Sets up all three Render services and GitHub Actions CI/CD pipeline. Leads OCR integration (Service 2), all four error detection modules, Flask API design, PostgreSQL schema, and python-docx/ReportLab export. Contributes equally to all development.

11. RUBRIC COMPLIANCE CHECKLIST

Requirement	How MediCheck Satisfies It	Owner	Sprint
GitHub repo with documented code	Public repo with docstrings; shared with quantic-grader day one	Member 2	1
Deployed web application URL	Three Render service URLs in README and deployed.md	Member 2	2
Agile task board	Trello Scrum Board: full product backlog, per-sprint backlogs, completion status	Member 1	1
Design & testing document	Architecture decisions, UML diagrams, pattern rationale, bounded contexts, deployment cost analysis, test results	Both	5

CI/CD tools	GitHub Actions: pytest + lint on PR; auto-deploy all three services on merge to main; /health smoke test post-deploy	Member 2	2
At least 3 sprints	6 sprints: 5 x 2-week + 1 x 1-week, with planning meetings and sprint demo recordings each	Both	Ongoing
15-20 min recorded presentation	Zoom: both members on camera; government IDs shown; full end-to-end demo; CI/CD pipeline shown running	Both	6
Agile methodology	Scrum: Product Owner, Scrum Master, Code Owner, sprint ceremonies, user stories, Definition of Done	Both	Ongoing
AI system (option a)	RAG + OCR + NLP + Strategy pattern + microservices — genuine AI engineering	Both	3-4
Goes above and beyond	Microservices architecture with DDD bounded contexts; dispute letter with regulatory citations; RAG evaluation metrics	Both	4-5
15+ user stories (Project 1)	Written from patient, admin, and developer personas with traceability matrix	Member 1	1
20+ requirements (Project 1)	Functional and non-functional, quantifiable, numbered, categorised	Member 1	1
Figma wireframes (Project 1)	Three core screens annotated with requirement IDs	Member 1	1
React + Flask + PostgreSQL (Project 2)	Service 1 (React), Service 2 (Flask + PostgreSQL), full-stack integration	Both	2
Design patterns + UML (Project 3)	Strategy pattern for detectors; Chain of Responsibility for RAG pipeline; 2x UML diagrams	Member 2	5
DDD + microservices diagram (Project 3)	Two bounded contexts; bounded context diagram; microservices architecture diagram	Both	5
Testing + CI/CD (Project 4)	Unit, integration, smoke tests; pytest; GitHub Actions gated deployment	Member 2	5
RAG evaluation (Project 5)	Groundedness, citation accuracy, latency; 15-20 question evaluation set	Member 1	5

12. PRESENTATION & DEMO PLAN

Segment	Duration	Presenter	What Is Shown

ID verification + intro	1 min	Both	Government IDs; project title; team names; Render URLs open in browser
Problem + architecture overview	2 min	Member 2	US billing problem; three-service architecture diagram; DDD bounded contexts explained
Repo + process evidence	2 min	Member 2	GitHub repo structure; Trello board; GitHub Actions CI/CD run showing tests and three-service deploy
Live demo: upload + field confirmation	3 min	Member 1	Upload sample bill PDF; Textract extraction; field confirmation UI; session stored in PostgreSQL
Live demo: error detection	3 min	Member 1	Analysis runs; all four modules fire; colour-coded error report with dollar impact shown
Live demo: RAG explanations + letter	3 min	Member 1	RAG grounded explanations shown with citations; Word and PDF dispute letter downloaded
Design decisions	2 min	Member 2	Strategy pattern; CPT licensing decision; microservices split rationale; deployment cost analysis
RAG evaluation results	1 min	Member 1	Groundedness %, citation accuracy %, p50/p95 latency shown
Wrap-up	1 min	Both	Future work; closing

Demo preparation: Prepare 2-3 sample bills with known errors before recording. Verify all three produce correct output. Warm up all three Render services by opening the app 2 minutes before recording — free tier services cold-start after 15 minutes of inactivity.

13. GEOGRAPHIC SCOPE & PROVIDER REFERENCE

MediCheck is scoped to the South Carolina and Charlotte-area North Carolina market for its initial demonstration. This two-state region was selected because it has a manageable number of dominant insurers, several large hospital systems whose billing patterns are well-documented, and a natural No Surprises Act test case — patients in South Carolina who receive care at North Carolina facilities across the state border.

All patient data in MediCheck sample and test documents is entirely synthetic and generated programmatically using the Python Faker library. Provider names and NPI numbers are publicly available from the CMS NPPES registry. Insurance company names are publicly known. No real patient records are used at any stage of development, testing, or demonstration.

> **Data compliance note:** This approach follows standard healthcare technology industry practice for development and demo environments. A disclaimer is included on all synthetic documents: 'All patient data is fictitious and generated for demonstration purposes only. These documents do not represent real patient records and are not derived from any real medical encounter.'

Geographic Coverage

State	Primary Cities / Regions	CMS Medicare Locality Code	Rationale
South Carolina	Charleston, Columbia, Greenville, Florence, Myrtle Beach	SC Locality 00 (statewide)	Home state; dominant insurers well-defined; MUSC and Prisma Health are major billing entities
North Carolina (Charlotte region)	Charlotte, Rock Hill border area	NC Locality 05 (Charlotte metro)	Atrium Health Carolinas Medical Center serves SC patients; ideal No Surprises Act crossborder scenario

Insurance Providers — Three Target Payers

Insurer	Plan Type	SC Coverage	Demo Scenario	EOB Notes
BlueCross BlueShield of South Carolina	PPO / EPO / HMO (BlueChoice)	All 46 SC counties — statewide dominant carrier	Scenario A: Employer-sponsored PPO plan. Patient crosses into NC for surgery. Anaesthesiologist out-of-network. No Surprises Act violation + duplicate charge + upcoding.	BCBS SC produces structured, consistent EOBs with clear key-value layout. Best template for Module 3 (EOB reconciliation). Payer ID: 38000.
Ambetter from Absolute Total Care	HMO (ACA Marketplace)	Statewide marketplace plan; competitive Silver tier	Scenario B: Marketplace HMO. Outpatient knee arthroscopy at MUSC Health Charleston. Separate radiology bill. EOB mismatch on	Ambetter EOBs are denser than BCBS. Good for demonstrating Module 3 mismatch between facility bill and insurer EOB. Payer ID: 68069.

			radiology procedure code.	
Molina Healthcare of South Carolina	HMO (Medicaid / Marketplace)	37 SC counties	Scenario C: Medicaid-adjacent marketplace plan. Prisma Health Medical Group primary care visit. Upcoded from established patient visit to new patient visit. Medicare rate outlier triggered.	Molina EOBs are straightforward single-page documents. Good for demonstrating Module 2 (rate outlier) in isolation. Payer ID: 38217.

Hospital Systems — Three Target Providers

System	Key Facility Used	Location	NPI (Public)	Demo Use
MUSC Health	MUSC Health University Medical Center	169 Ashley Ave, Charleston, SC 29425	1437158858	Primary facility for Scenario B. Outpatient surgical facility bill. Separate anaesthesiology group bill triggers NSA evaluation.
Prisma Health	Prisma Health Greenville Memorial Hospital	701 Grove Rd, Greenville, SC 29605	1790786653	Primary facility for Scenario C. Outpatient physician visit. Upcoding scenario using Prisma Health Medical Group as the billing physician entity.
Atrium Health (Advocate Health)	Atrium Health Carolinas Medical Center	1000 Blythe Blvd, Charlotte, NC 28203	1174675214	Primary facility for Scenario A. SC patient treated at NC facility. Generates crossborder No Surprises Act scenario with out-of-network anaesthesiologist.

NPI numbers: All NPI numbers above are publicly available from the CMS National Plan and Provider Enumeration System (NPPES) at nppes.cms.hhs.gov. They are used solely to produce realistic-looking synthetic documents and do not imply any relationship with or endorsement by these healthcare organisations.

Outpatient / Specialist Providers

Provider Type	Synthetic Entity Name Used	Billing Scenario	Error Triggered
Anaesthesiology group	Carolina Anesthesia Associates, PA	Separate bill for anaesthesia at Atrium Health (Scenario A). Provider is out-of-network with BCBS SC.	Module 4 — No Surprises Act violation
Radiology group	Lowcountry Radiology Consultants, LLC	Separate bill for imaging services at MUSC Health (Scenario B). CPT code on bill differs from EOB.	Module 3 — EOB reconciliation mismatch

Emergency medicine group	Palmetto Emergency Physicians, PC	Separate bill for EM physician services at in-network hospital. Provider out-of-network.	Module 4 — No Surprises Act violation (secondary scenario)
Primary care physician	Dr. Sarah M. Harrington, MD (Prisma Health Medical Group)	Outpatient office visit (Scenario C). Billed as new patient high-complexity; records indicate established patient routine visit.	Module 2 — Medicare rate outlier (upcoding indicator)

Three Demo Scenarios — Summary

Scenario	Patient Profile	Insurer	Provider	Errors Injected	Modules Triggered
A — Crossborder Surgery	Male, 45, Rock Hill SC. Employer-sponsored BCBS SC PPO. Elective knee surgery.	BlueCross BlueShield of South Carolina	Atrium Health Carolinas Medical Center (Charlotte NC) + Carolina Anesthesia Associates (out-of-network)	1. Surgical tray billed twice ($480 duplicate). 2. Anaesthesia time upcoded by 90 min ($1,100 overcharge). 3. Out-of-network anaesthesiologist (No Surprises Act violation).	All four modules (1, 2, 3, 4)
B — Outpatient Arthroscopy	Female, 38, Charleston SC. Ambetter HMO marketplace plan. Outpatient knee arthroscopy.	Ambetter from Absolute Total Care	MUSC Health University Medical Center + Lowcountry Radiology Consultants	1. Radiology CPT code on provider bill (72193) differs from EOB (72191 — different imaging protocol). 2. MUSC facility fee at 340% of Medicare rate ($4,200 billed vs $1,235 Medicare rate).	Modules 2 and 3
C — Primary Care Upcoding	Female, 52, Greenville SC. Molina Healthcare HMO. Routine diabetes follow-up visit.	Molina Healthcare of South Carolina	Prisma Health Medical Group (Dr. Sarah M. Harrington, MD)	1. Visit billed as CPT 99205 (new patient, high complexity, $280) when patient is established and visit was routine follow-up — should be CPT 99213 ($87). Billed amount is 322% of Medicare rate for correct code.	Module 2 (rate outlier / upcoding indicator)

14. DIFFERENTIATION FROM EXISTING TOOLS

Feature	MediCheck	Professional Auditor	Generic AI Chatbot
Cost to user	Free	$500–$2,000 upfront	Free but no domain grounding
Analyses EOB + bill together	Yes — cross-references both	Yes	No structured document handling
Detects No Surprises Act violations	Yes — RAG-grounded	Yes	May hallucinate
Generates dispute letter	Yes — regulatory citations included	Yes	Ungrounded, risky to use
Accessible to non-experts	Yes — plain English output	No	Partially
Hallucination-safe	Yes — deterministic detection layer	Yes	No
Microservices architecture	Yes — three bounded services	N/A	N/A

MediCheck delivers in 30 seconds what currently costs $500–$2,000 — built as a properly architected microservices AI system across 11 weeks part-time, with real-world financial impact for a large addressable audience, deployed end-to-end on a modern cloud platform.